Celebal Summer Internship 2026

Final Project Report

Project Title: Customer Support Ticket Resolution Pipeline

Submission Date:

12-07-2026

Prepared By:

Keshav Kumar sharma

Project Overview

The **Customer Support Ticket Resolution Pipeline** is an end-to-end Azure Data Engineering project developed to process customer support ticket data stored in Azure Data Lake Storage Gen2 (ADLS Gen2). Using Azure Databricks (PySpark), the pipeline ingests raw CSV files, performs data validation and transformation, applies business rules, and generates Gold Layer datasets. These datasets are finally visualized in Power BI to provide meaningful insights into team and agent performance.

Problem Statement

Customer support ticket data is collected daily in CSV format. However, the raw data contains missing values, inconsistent records, unresolved tickets, and resolution time stored as text, making direct analysis difficult. The goal of this project is to automate the complete data processing workflow and generate clean, business-ready datasets for reporting and decision-making.

Proposed Solution

To solve these challenges, an end-to-end Medallion Architecture pipeline was implemented using Azure services

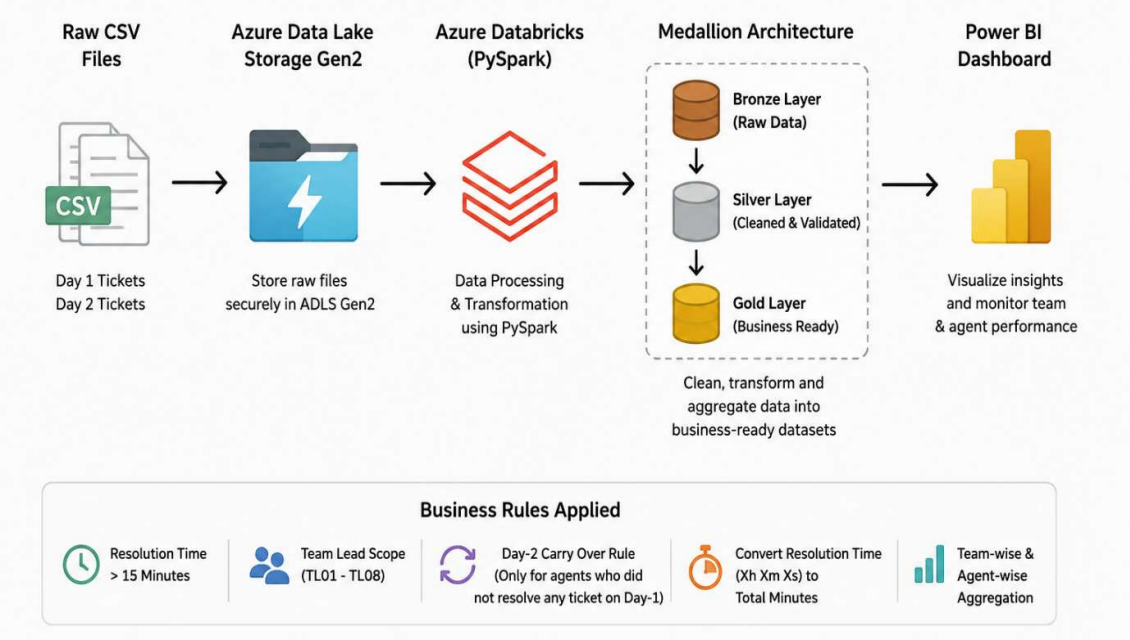
The pipeline:

- Reads raw CSV files from Azure Data Lake Storage Gen2.
- Validates and cleans inconsistent records.
- Converts resolution time from text format into numeric minutes.
- Applies all required business rules.
- Creates business-ready Gold Layer datasets.
- Generates interactive Power BI dashboards for performance monitoring.

This solution provides accurate KPIs and enables leadership to monitor customer support efficiency at both the team and agent levels.

System Architecture

Below is the overall workflow of the Customer Support Ticket Resolution Pipeline.



Pipeline Implementation

Step 1 : Uploading Raw Dataset to ADLS Gen2 Description

The generated customer support datasets were first uploaded to the **Raw** container in Azure Data Lake Storage Gen2. These files serve as the original source data for the entire pipeline.

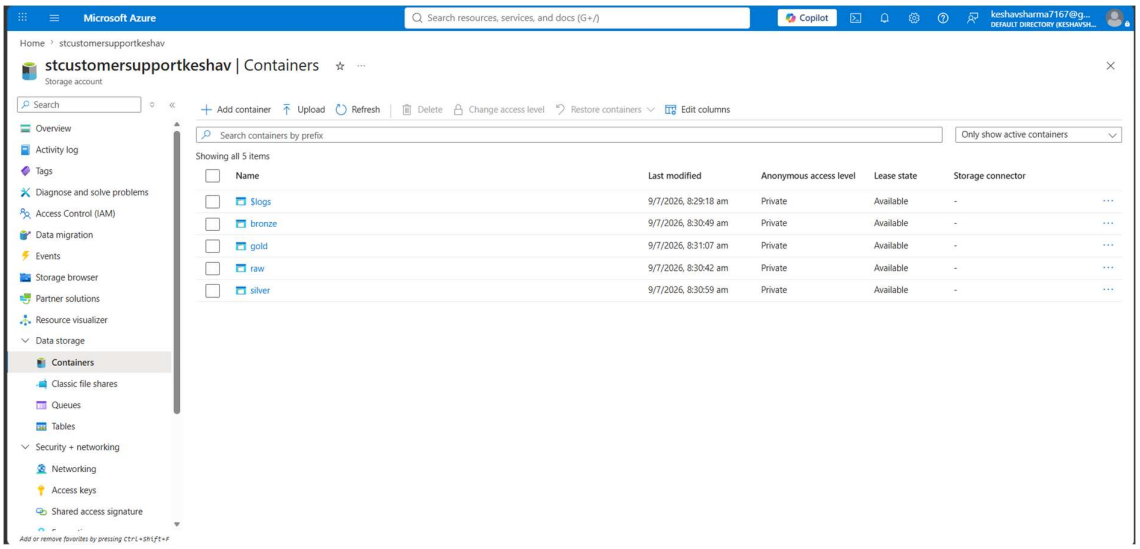


Fig: ADLS Gen2 Container Structure

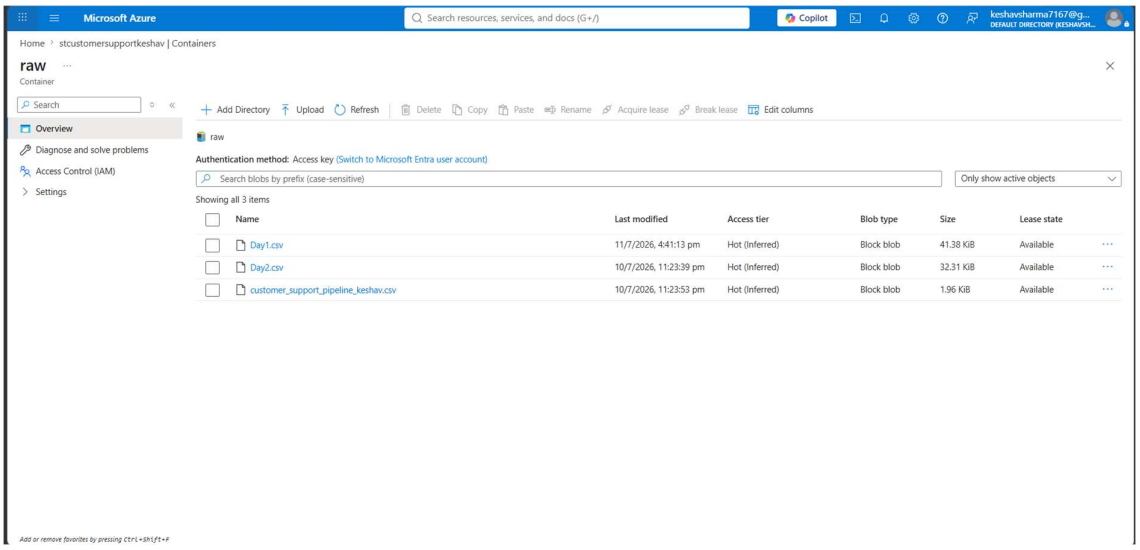


Fig : Raw customer support datasets uploaded to the Raw container.

Step 2 — Azure Data Factory Data Ingestion

Azure Data Factory was used to automate data ingestion. Three Copy Data activities were created to copy the Agent Profile, Day 1 Tickets, and Day 2 Tickets datasets from the Raw container to the Bronze container. This establishes the Bronze layer of the Medallion Architecture.

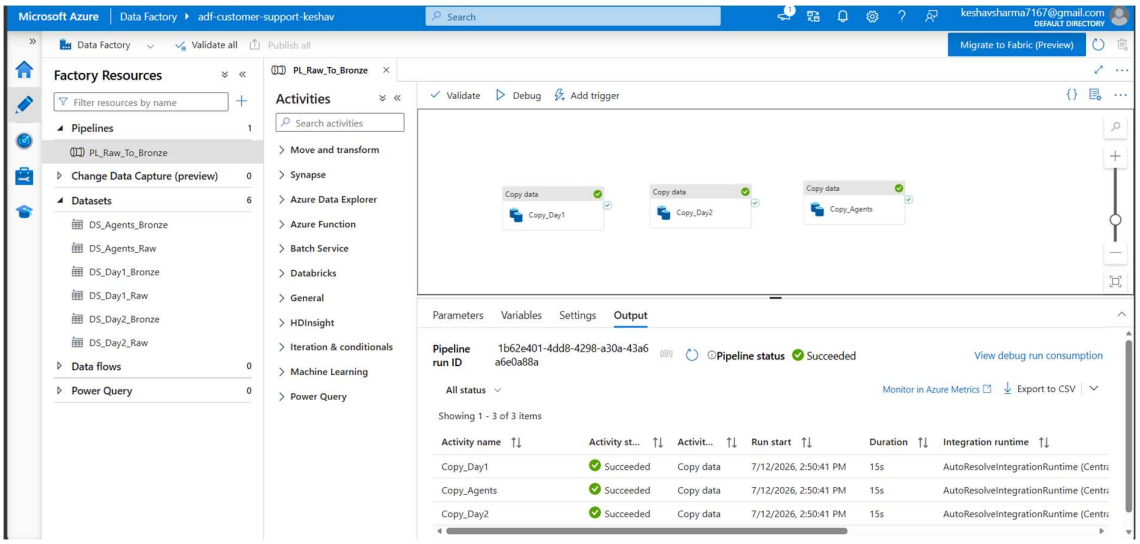


Fig : Azure Data Factory pipeline copying raw datasets into the Bronze layer.

Step 3 — Bronze Layer Processing

The Bronze datasets stored in ADLS Gen2 were accessed using Azure Databricks. The CSV files were loaded into PySpark DataFrames with schema inference enabled. An additional Day column was added to distinguish records from Day 1 and Day 2, preparing the datasets for downstream processing.

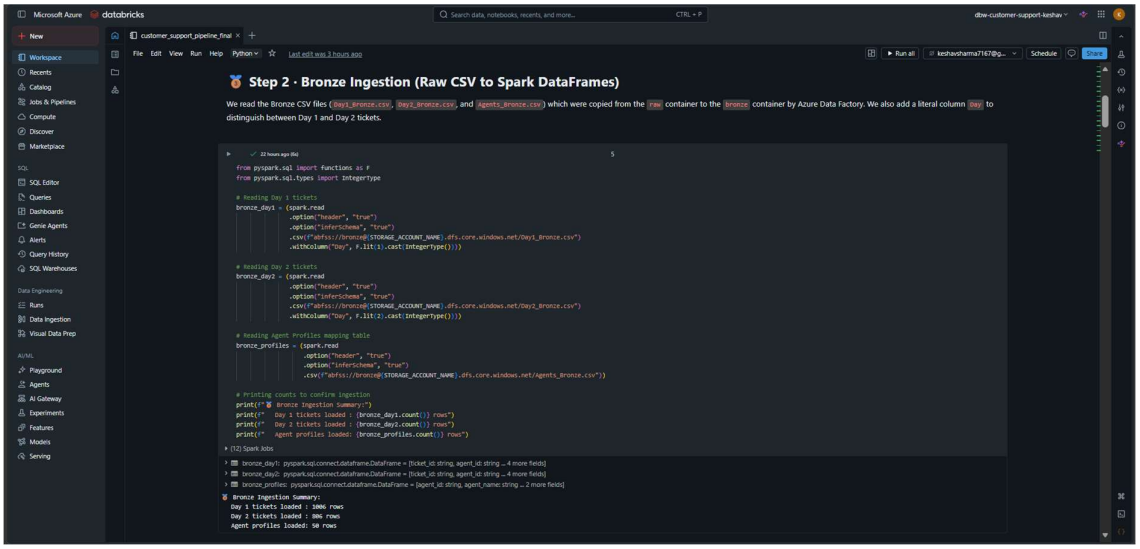


Fig : Reading Bronze layer datasets into PySpark DataFrames.

Step 4 – Silver Layer (Cleaning, Processing & Filtering)

The Silver layer is responsible for cleaning and transforming the Bronze data into a structured and reliable dataset. At this stage, multiple business rules were applied to improve data quality and prepare the data for analytical processing.

Business Rules Applied

Rule	Description
R-1 & R-2	Convert resolution time from text to numeric minutes with rounding.
R-3	Keep only resolved tickets with resolution time greater than 15 minutes.
R-4	Filter records belonging only to Team Leads TL01–TL08.
R-5	Remove records with missing ticket ID, agent ID, or resolution time.

4.1 Data Quality Validation

The pipeline validates critical fields such as ticket_id, agent_id, and resolution_time. Records containing null or empty values are removed before further processing.

```
=== Data Quality Summary ===
Day 1 Before Cleaning : 1006
Day 1 After Cleaning  : 1006
Day 2 Before Cleaning : 806
Day 2 After Cleaning  : 806
```

Table +

	ticket_id	agent_id	resolution_time
1	TKT00001	A036	0h 30m 9s
2	TKT00002	A042	1h 17m 11s
3	TKT00003	A004	0h 44m 43s
4	TKT00004	A005	0h 28m 27s
5	TKT00005	A039	0h 49m 27s
6	TKT00006	A011	1h 13m 32s
7	TKT00007	A007	2h 53m 16s
8	TKT00008	A039	2h 2m 11s
9	TKT00009	A026	1h 3m 48s
10	TKT00010	A021	1h 50m 15s

Fig : Data Quality Validation Results.

4.2 Resolution Time Conversion & Rounding

The pipeline converts the resolution_time values from the "Xh Xm Xs" format into total minutes using a custom PySpark UDF. It also applies the required rounding rule, where values with 30 or more seconds are rounded up to the next minute.

Table +

	resolution_time	converted_minutes
1	0h 30m 9s	30
2	1h 17m 11s	77
3	0h 44m 43s	45
4	0h 28m 27s	28
5	0h 49m 27s	49
6	1h 13m 32s	74
7	2h 53m 16s	173
8	2h 2m 11s	122
9	1h 3m 48s	64
10	1h 50m 15s	110

Fig: Resolution Time Parsing and Rounding using PySpark UDF

4.3 Applying Resolution Time Transformation

After defining the UDF, it is applied to the Day 1 and Day 2 datasets to generate a new **resolved_minutes** column. This transformed data is used in subsequent filtering, quality validation, and business-rule processing.

Table +

	ticket_id	resolution_time	resolved_minutes
1	TKT00001	0h 30m 9s	30
2	TKT00002	1h 17m 11s	77
3	TKT00003	0h 44m 43s	45
4	TKT00004	0h 28m 27s	28
5	TKT00005	0h 49m 27s	49
6	TKT00006	1h 13m 32s	74
7	TKT00007	2h 53m 16s	173
8	TKT00008	2h 2m 11s	122
9	TKT00009	1h 3m 48s	64
10	TKT00010	1h 50m 15s	110

10 rows | 9.427s runtime

Fig: Resolution Time Transformation Applied to Ticket Data

4.4 Team Lead Scope Filtering

The pipeline joins the ticket data with the agent profile dataset to retrieve agent information such as agent name, role, and team lead. After enrichment, only agents reporting to Team Leads TL01 through TL08 are retained. This ensures that the analysis is limited to the defined business scope.

Table +

	agent_id	agent_name	team_lead_id	role
1	A036	Agent_A036	TL06	Senior Support Age...
2	A042	Agent_A042	TL07	Junior Support Agent
3	A004	Agent_A004	TL01	Senior Support Age...
4	A005	Agent_A005	TL01	Senior Support Age...
5	A039	Agent_A039	TL07	Support Agent
6	A011	Agent_A011	TL02	Support Agent
7	A007	Agent_A007	TL02	Senior Support Age...
8	A039	Agent_A039	TL07	Support Agent
9	A026	Agent_A026	TL05	Senior Support Age...
10	A021	Agent_A021	TL04	Senior Support Age...

Fig : Scope Filtering (TL01–TL08)

4.5 Quality Threshold Validation

The pipeline applies the business quality rule that a ticket is considered successfully resolved only if its status is RESOLVED and the resolution time is greater than 15 minutes. Tickets that do not satisfy this condition are excluded from further analysis.

	ticket_id	agent_id	status_clean	resolved_minutes
1	TKT00002	A042	RESOLVED	77
2	TKT00005	A039	RESOLVED	49
3	TKT00007	A007	RESOLVED	173
4	TKT00008	A039	RESOLVED	122
5	TKT00011	A014	RESOLVED	54
6	TKT00012	A014	RESOLVED	61
7	TKT00014	A005	RESOLVED	19
8	TKT00016	A026	RESOLVED	35
9	TKT00018	A048	RESOLVED	125
10	TKT00021	A023	RESOLVED	159

10 rows | 9.854s runtime

Fig : Quality Threshold Validation (>15 Minutes)

4.6 Day-2 Carry-over Rule

The pipeline applies the Day-2 Carry-over Rule to prevent double-counting of agents. All agents who successfully resolved tickets on Day 1 are excluded from the Day 2 dataset using a Left Anti Join. The remaining Day 2 records are then combined with the qualified Day 1 records to create the final business-ready dataset.

```
=== Carry Over Summary ===
Day 1 Successful Agents : 45
Day 2 Carry Over Records : 12
Final Gold Dataset Rows  : 505
```

Table +

	agent_id	agent_name	team_lead_id	resolved_minutes
1	A004	Agent_A004	TL01	27
2	A004	Agent_A004	TL01	172
3	A021	Agent_A021	TL04	33
4	A021	Agent_A021	TL04	160
5	A004	Agent_A004	TL01	36
6	A021	Agent_A021	TL04	106
7	A021	Agent_A021	TL04	112
8	A004	Agent_A004	TL01	51
9	A011	Agent_A011	TL02	34
10	A011	Agent_A011	TL02	54

Fig : Day-2 Carry-over Rule Implementation

Question 1 : Team-wise Ticket Resolution

To evaluate team performance, the pipeline aggregates the final Gold dataset by Team Lead. It calculates the total number of resolved tickets, the number of active agents, and the average number of resolved tickets per agent. This provides a clear comparison of team performance across all Team Leads.

```
> gold_q1: pyspark.sql.connect.dataframe.DataFrame = [team_lead_id: string, total_resolved_tickets: long ... 2 more fields]
Q1 Table Preview:
+------------+----------------------+-------------+--------------------+
|team_lead_id|total_resolved_tickets|active_agents|avg_tickets_per_agent|
+------------+----------------------+-------------+--------------------+
|        TL05|                    74|            6|               12.33|
|        TL03|                    73|            6|               12.17|
|        TL06|                    72|            6|                12.0|
|        TL07|                    68|            6|               11.33|
|        TL01|                    57|            6|                 9.5|
|        TL08|                    56|            6|                9.33|
|        TL02|                    56|            6|                9.33|
|        TL04|                    49|            6|                8.17|
+------------+----------------------+-------------+--------------------+
```

Fig : Gold Layer – Team-wise Aggregation (Question 1)

Question 2 : Per-Agent Performance Analysis

The pipeline analyzes the performance of each support agent across Day 1 and Day 2. It calculates the number of resolved tickets on each day, the total resolved tickets, and identifies the performance trend (such as Improved, Declined, Stable, Day 1 Only, or Day 2 Carry-over). This helps leadership evaluate individual agent performance over the two-day review period.

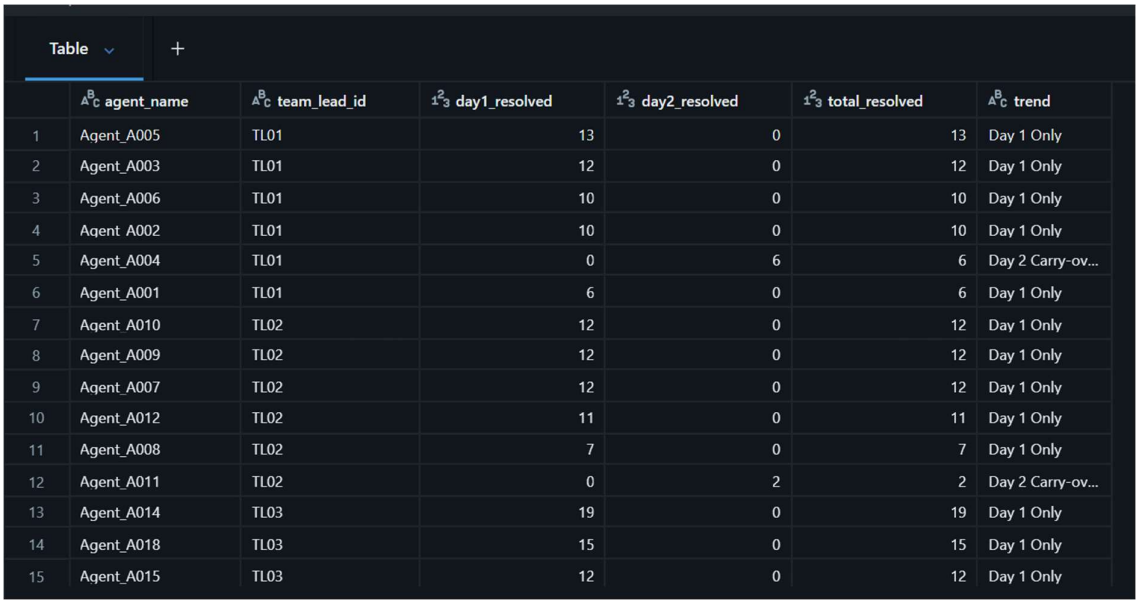


Table +

	agent_name	team_lead_id	day1_resolved	day2_resolved	total_resolved	trend
1	Agent_A005	TL01	13	0	13	Day 1 Only
2	Agent_A003	TL01	12	0	12	Day 1 Only
3	Agent_A006	TL01	10	0	10	Day 1 Only
4	Agent_A002	TL01	10	0	10	Day 1 Only
5	Agent_A004	TL01	0	6	6	Day 2 Carry-ov...
6	Agent_A001	TL01	6	0	6	Day 1 Only
7	Agent_A010	TL02	12	0	12	Day 1 Only
8	Agent_A009	TL02	12	0	12	Day 1 Only
9	Agent_A007	TL02	12	0	12	Day 1 Only
10	Agent_A012	TL02	11	0	11	Day 1 Only
11	Agent_A008	TL02	7	0	7	Day 1 Only
12	Agent_A011	TL02	0	2	2	Day 2 Carry-ov...
13	Agent_A014	TL03	19	0	19	Day 1 Only
14	Agent_A018	TL03	15	0	15	Day 1 Only
15	Agent_A015	TL03	12	0	12	Day 1 Only

Fig : Gold Layer – Per-Agent Performance (Question 2)

Question 3 : Compliance Rate Analysis

The pipeline evaluates the resolution quality compliance for each Team Lead by comparing the number of tickets that passed the quality threshold (resolution time > 15 minutes) with the total number of resolved tickets. The compliance percentage helps identify teams that consistently meet the organization's quality standards.

Table +

	team_lead_id	qualifying_tickets	total_resolved_any_time	compliance_rate_pct
1	TL05	74	110	67.27
2	TL03	73	119	61.34
3	TL07	68	116	58.62
4	TL06	72	127	56.69
5	TL01	57	102	55.88
6	TL02	56	109	51.38
7	TL08	56	113	49.56
8	TL04	49	99	49.49

8 rows | 5.828s runtime

Fig : Gold Layer – Compliance Rate Analysis (Question 3)

Question 4 : Carry-over Agent Analysiss

The pipeline identifies agents who did not successfully complete their work on Day 1 but resolved qualifying tickets on Day 2. According to the business rule, only these carry-over agents are included in the Day 2 analysis, preventing successful Day 1 agents from being counted twice.

The output contains each carry-over agent along with the number of qualifying Day 2 tickets, average resolution time, minimum resolution time, and maximum resolution time. This helps management monitor pending workloads and evaluate how efficiently unresolved work was completed on the following day.

```
Q4 Table Preview (Top 10 rows):
+--------+----------+------------+----------------------+----------------------+------------------+------------------+
|agent_id|agent_name|team_lead_id|day2_qualifying_tickets|avg_resolution_mins_day2|min_resolution_mins|max_resolution_mins|
+--------+----------+------------+----------------------+----------------------+------------------+------------------+
|    A004|Agent_A004|        TL01|                     6|                  59.0|                18|               172|
|    A011|Agent_A011|        TL02|                     2|                  44.0|                34|                54|
|    A021|Agent_A021|        TL04|                     4|                102.75|                33|               160|
+--------+----------+------------+----------------------+----------------------+------------------+------------------+
```

Fig : Gold Layer – Carry-over Agent Analysis (Question 4)

Step 5 - Power BI Dashboard

The final Gold Layer datasets were imported into **Power BI** to create an interactive dashboard for analyzing customer support performance. The dashboard provides key insights through KPI cards, charts, and tables, with a **Team Lead** filter for interactive analysis.

The dashboard includes:

- Total Resolved Tickets
- Average Compliance Percentage
- Carry-over Agents
- Active Agents
- Resolved Tickets by Team Lead
- Compliance Rate by Team Lead
- Agent-wise Day 1 vs Day 2 Performance
- Carry-over Agent Details
- Team Lead Filter (Slicer)

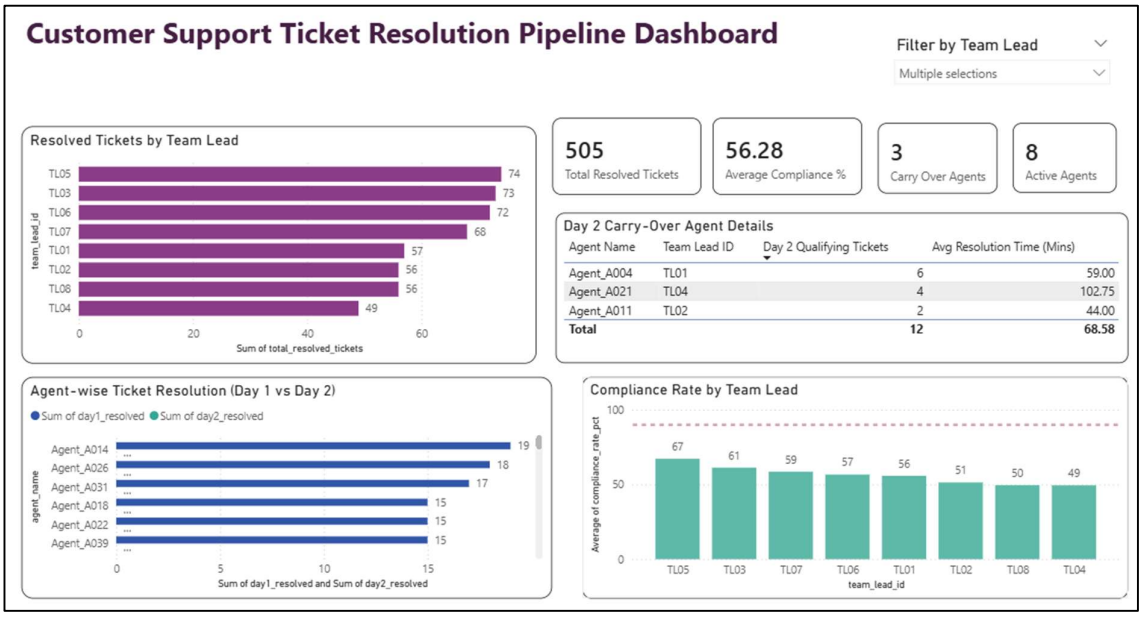


Fig : Power Bi Dashboard

Conclusion

The Customer Support Ticket Resolution Pipeline was successfully implemented using Azure Data Factory, Azure Data Lake Storage Gen2, Azure Databricks (PySpark), Delta Lake, and Power BI. The pipeline cleaned, validated, transformed, and aggregated raw ticket data according to the defined business rules. The final Power BI dashboard provides interactive insights into team performance, compliance, and carry-over analysis, helping management make informed decisions.

Project Outcomes

The project successfully achieved the following objectives:

- Built an end-to-end Azure Data Engineering pipeline.
- Implemented the Bronze → Silver → Gold Medallion Architecture.
- Applied all required business rules for data validation and transformation.
- Generated four business-ready Gold Layer datasets.
- Created an interactive Power BI dashboard for business insights.

Future Enhancements

The pipeline can be further enhanced by automating the daily data ingestion and processing workflow using Azure Data Factory Triggers. This will allow the pipeline to run automatically whenever new ticket files are available, eliminating manual execution and improving operational efficiency.